Propagation-Based Vulnerability Impact Assessment for Software Supply Chains

Bonan Ruan Zhiwei Lin Jiahao Liu* Chuqi Zhang Kaihang Ji Zhenkai Liang
National University of Singapore
{r-bonan, zhiweil, jiahao99, chuqiz, kaihang, liangzk}@comp.nus.edu.sg

arXiv:2506.01342v2 [cs.SE] 9 Oct 2025

***Abstract*—Identifying the impact scope and scale is critical for software supply chain vulnerability assessment. However, existing studies face substantial limitations. First, prior studies either work at coarse package-level granularity—producing many false positives—or fail to accomplish whole-ecosystem vulnerability propagation analysis. Second, although vulnerability assessment indicators like CVSS characterize individual vulnerabilities, no metric exists to specifically quantify the dynamic impact of vulnerability propagation across software supply chains. To address these limitations and enable accurate and comprehensive vulnerability impact assessment, we propose a novel approach: (i) a hierarchical worklist-based algorithm for whole-ecosystem and call-graph-level vulnerability propagation analysis and (ii) the Vulnerability Propagation Scoring System (VPSS), a dynamic metric to quantify the scope and evolution of vulnerability impacts in software supply chains. We implement a prototype of our approach in the Java Maven ecosystem and evaluate it on 100 real-world vulnerabilities. Experimental results demonstrate that our approach enables effective ecosystem-wide vulnerability propagation analysis, and provides a practical, quantitative measure of vulnerability impact through VPSS.**

I. Introduction

The proliferation of software vulnerabilities has introduced significant security risks [1]. However, not all vulnerabilities carry the same impact scope. Specifically, a vulnerability in a client application usually only affects the application itself, while a vulnerability in software supply chains often puts downstream software that depends on the vulnerable upstream libraries directly or transitively at risk as well. For instance, *Heartbleed* [2], a vulnerability in the OpenSSL library, affects countless services that rely on it. Another vulnerability, *Log4Shell*, endangers numerous projects depending on the popular Apache Log4j logging framework [3]. Hence, it is critical to *identify the impact scope and scale of the vulnerability in software supply chains* after a vulnerability is disclosed, which is called *vulnerability propagation analysis*.

Researchers have conducted several studies to investigate this problem for popular programming language ecosystems (*e.g.,* Java [4]–[18], JavaScript [19]–[23], and Python [24]) by analyzing components and the corresponding dependencies in software supply chains. While existing works take significant steps toward software supply chain vulnerability analysis, a substantial gap remains in enabling accurate and comprehensive impact assessment. We observe the following fundamental limitations that render current solutions suboptimal:

Limitation 1 (L1): The lack of accurate and complete vulnerability propagation analysis. First, many studies only conduct package-level vulnerability propagation analysis based on dependency declarations. This often leads to false positives, as downstream projects may declare dependencies on vulnerable packages without actually invoking the vulnerable functions (VFs) [11], [14], [18]. Second, although the remaining studies have explored call graph (CG)-level analysis, their methods are often limited in scope and incomplete due to a lack of efficient processing techniques for complex whole-ecosystem dependencies. Specifically, (i) they consider only partial dependency relationships rather than the complex, ecosystem-wide structure; (ii) they analyze only a subset of project versions or a limited number of downstream projects, instead of covering all relevant versions and dependencies; and (iii) they focus solely on direct dependencies, ignoring vulnerability propagation through transitive dependencies. We provide a detailed discussion of these limitations in § II-B1.

Limitation 2 (L2): The lack of metrics for quantifying the impact of vulnerabilities across software supply chains. The widely adopted vulnerability assessment indicators are only used to characterize the impact of a vulnerability itself. For instance, although people can perceive the severity of a vulnerability through its CVSS score [25], it is fundamentally designed to assess and reflect the characteristics of individual vulnerabilities. As such, its application does not extend well to measuring vulnerability impacts across software supply chains, which is explicitly acknowledged in the CVSS v4.0 FAQ [25].

In this work, to address the aforementioned limitations, we propose a novel approach to accurate and whole-ecosystem impact assessment of software supply chain vulnerabilities and a propagation-based indicator for quantifying such impact.

- To address **L1**, we draw inspiration from data-flow analysis [26], [27] and design a worklist-based vulnerability propagation analysis algorithm to efficiently identify affected downstream dependencies given a vulnerability. The algorithm conducts CG-level analysis that considers complete dependency relations, all potentially affected downstreams, and transitive dependencies across the entire ecosystem. We integrate hierarchical pruning strategies into the propagation algorithm to reduce the complexity of analysis.
- To address **L2**, we propose the *Vulnerability Propagation Scoring System* (VPSS), a graph-theoretic dynamic indicator specialized for quantifying vulnerability impact in software supply chains and reflecting the temporal evolution of im-

*Corresponding author

TABLE I: Term Unification Across Ecosystems

Ecosystem	Project (*P*)	Project-Version (*PV*)
Maven (Java)	GroupId:ArtifactId	GroupId:ArtifactId:Version
npm (JavaScript)	package name	package@version
PyPI (Python)	distribution name	name==version

pact. It is designed to consider both the breadth and depth of vulnerability propagation. VPSS has a similar score range (0–10) and impact levels (low, medium, high, and critical) as CVSS [25], making it easy to understand and use.

We implement a prototype of our approach for the Java Maven ecosystem [28], one of the largest software ecosystems in the world, and evaluate it on 100 real-world vulnerabilities investigated by prior work [14]. To assess our approach, we ask and answer two evaluation questions:

- *How effective and scalable is our ecosystem-scale vulnerability propagation analysis in identifying affected downstream projects?*
  Findings. Our approach successfully and efficiently completes the propagation analysis for all 100 vulnerabilities. On average, 97.8% of projects and 99.2% of project version releases are pruned during the analysis, with the longest and average propagation path lengths reduced by at least 34.1% and 29.4%, respectively. Importantly, our approach significantly lowers the cost of CG construction by reducing the number of CGs that need to be built.
- *What insights can be drawn from the VPSS scores?*
  Findings. The computed VPSS scores generally decline over time after disclosure, driven by patch adoption and ecosystem expansion. Interestingly, some CVEs (*e.g.,* CVE-2016-3086 [29]) show temporary score increases due to delayed dependency updates. Across the entire dataset, VPSS scores remain relatively low and gradually stabilize. This aligns with expected ecosystem dynamics where vulnerability propagation attenuates over time.

To the best of our knowledge, this is the first work achieving CG-level and whole-ecosystem vulnerability impact assessment for software supply chains. In summary, this paper makes the following contributions:

- We design a hierarchical worklist-based vulnerability propagation analysis algorithm to accurately and efficiently identify affected downstream dependencies across a whole software ecosystem.
- We propose Vulnerability Propagation Scoring System (VPSS), the first time-aware indicator for quantifying vulnerability impact in software supply chains.
- We implement a prototype of our approach for the Java Maven ecosystem and evaluate it on real-world vulnerabilities. The code and experimental dataset are available at https://github.com/brant-ruan/vpss.

II. Background and Motivations

A. Background

1) Terminology: Different software ecosystems use diverse naming conventions to refer to software units and their versions, such as *packages*, *modules*, or *distributions*. This inconsistency may lead to inaccurate descriptions and hinder understanding. To provide a unified abstraction across ecosystems, we use the terms *Project* (*P*) and *Project-Version* (*PV*) to represent software units and their specific releases, respectively. Table I shows how these terms correspond to identifiers in representative ecosystems.

2) Vulnerability Propagation Analysis: Given a vulnerability, identifying the scope and scale of its impact in software supply chains is called *vulnerability propagation analysis*, which takes vulnerability intelligence and inter-project dependencies as input to reason out the vulnerability's impact on downstream projects. Vulnerability propagation analysis can be conducted at different granularity levels. One option is the *PV*-level analysis, which considers a downstream *PV* as affected by the vulnerability if it declares a dependency on the upstream vulnerable *PV*s. The other one is the call graph (CG)-level analysis, which regards a downstream *PV* as affected only when it directly or transitively calls vulnerable functions (VFs) of the upstream vulnerable *PV*s. A prerequisite for CG-level analysis is to identify the VFs, where the vulnerable logic exists. Currently, the most widely adopted VF identification method is the patch-based approach [8], [14], [30]–[32], which identifies functions deleted or modified in patches as VFs, a widely adopted strategy due to its logical rationale and alignment with standardized patch information.

B. Limitations of Existing Solutions

1) Vulnerability Propagation Analysis: To better profile existing vulnerability propagation research, we conduct a comprehensive literature review. Prior works are mainly empirical studies on Java [4]–[18], JavaScript [19]–[23], and Python [24] ecosystems, as shown in Table II. To clearly display and compare these works, we profile them from six aspects:

Direction indicates whether the work conducts a forward analysis to answer *'which vulnerabilities in upstream dependencies affect a downstream project'*, or a backward analysis to answer *'which downstream projects (as the call sites) are affected by an upstream vulnerability—by using a chain of function calls to reach it'*. Intuitively, backward analysis is more suitable for vulnerability impact analysis, as it starts from the vulnerable site and propagates to the downstream projects.

Dep Scope clarifies whether the work examines the partial or complete dependency relations for a target ecosystem. For example, if a work only selects a subset from a software ecosystem with their dependencies for analysis, it has partial dependency scope. Instead, if the work conducts the propagation analysis with considering the whole ecosystem and dependency relations, the dependency scope is complete.

Coverage shows whether the work analyzes partial or complete projects. For example, if a work only selects one version to represent the target project, then it has partial coverage. Conversely, if the work considers all released versions for the propagation, it has complete coverage.

Transitivity indicates whether the work analyzes only the direct dependency relations or the transitive dependencies.

TABLE II: Summary of related works in comparison with this work. In 'LAN', 'JA' stands for Java, 'JS' stands for JavaScript, and 'PY' stands for Python. In 'Granularity', 'PV' means the work analyzes the propagation at *PV* level, and 'CG' means CG-level analysis. In 'VF Identification', 'Manual' means the work identifies VFs manually, 'Patch' means the work uses a patch-based method to identify VFs, and 'Patch (Optimized)' means the work uses an optimized patch-based method.

Year	LAN	Research	Direction	Dep Scope	Coverage	Transitivity	Granularity	VF Identification
2015	JA	Cadariu *et al.* [4]	Forward	Partial	Partial	Direct	PV	✗
2015	JA	Ponta *et al.* [5]	Forward	Partial	Partial	Direct	PV	Patch
2017	JS	Lauinger *et al.* [19]	Forward	Partial	Partial	Transitive	PV	✗
2018	JS	Decan *et al.* [20]	Forward	Partial	Partial	Direct	PV	✗
2018	JA	Kula *et al.* [6]	Forward	Partial	Partial	Direct	PV	✗
2018	JA	Du *et al.* [7]	Forward	Partial	Partial	Direct	PV	✗
2018	JA	Ponta *et al.* [8]	Forward	Partial	Partial	Direct	CG	Patch
2018	JA	Pashchenko *et al.* [9]	Forward	Partial	Partial	Direct	PV	Patch
2019	JA	Hu *et al.* [10]	Forward	Partial	Complete	Transitive	PV	✗
2019	JS	Zimmermann *et al.* [21]	Backward	Complete	Complete	Transitive	PV	✗
2020	JA	Wang *et al.* [11]	Forward	Partial	Partial	Direct	CG	Patch
2020	JA	Ponta *et al.* [12], [33]	Forward	Partial	Partial	Direct	CG	Patch
2020	PY	Ma *et al.* [24]	Backward	Partial	Partial	Transitive	CG	Manual
2022	JS	Liu *et al.* [23]	Backward	Complete	Complete	Transitive	PV	✗
2023	JS	Wang *et al.* [22]	Backward	Complete	Complete	Transitive	PV	✗
2023	JA	Zhang *et al.* [13]	Backward	Complete	Complete	Transitive	PV	✗
2023	JA	Wu *et al.* [14]	Forward	Complete	Partial	Direct	CG	Patch
2023	JA	Mir *et al.* [15]	Forward	Partial	Complete	Transitive	CG	Patch
2024	JA	Ma *et al.* [16]	Forward	Partial	Complete	Transitive	PV	✗
2024	JA	Zhang *et al.* [17]	Backward	Partial	Complete	Direct	CG	Patch (Optimized)
2025	JA	Shen *et al.* [18]	Backward	Partial	Partial	Transitive	CG	Patch
		This Work	Backward	Complete	Complete	Transitive	CG	Patch (Optimized)

Direct analysis only takes projects that directly depend on the vulnerable project into consideration, *i.e.,* one-hop dependency; transitive analysis considers multi-hop dependency towards the root vulnerable project.

Granularity shows the granularity at which a work conducts the propagation analysis. CG-level analysis inspects whether downstream *PV*s directly or transitively call upstream VFs, which is much more accurate than *PV*-level analysis that only considers *PV* dependency relations [14].

VF identification indicates whether the work identifies VFs and how it identifies them. Manual identification cannot be scaled to large ecosystems. Although the patch-based method has been widely used in existing work, it has two limitations. First, patches are not always publicly available [34], though several methods have been proposed to address this [34]–[39], which are beyond the scope of this paper. Second, patches sometimes contain changes unrelated to the vulnerability.

2) Vulnerability Assessment: For a software supply chain vulnerability, it is equally important to assess its own characteristics and its impact on downstream dependents. The Common Vulnerability Scoring System (CVSS) provides a way to capture the principal characteristics of a vulnerability and produce a numerical score (0–10) reflecting its severity [40]. However, it is fundamentally designed to assess and reflect the characteristics and severity of individual vulnerabilities, and does not extend well to measuring vulnerability impacts across software supply chains. This limitation is explicitly acknowledged in the CVSS v4.0 FAQ [25], where it is clarified that *there is no prescribed way to use CVSS Base and Environmental metrics to score a vulnerability along a long supply chain*. Furthermore, prior research mainly focuses on automating existing assessments [41]–[51] or proposing new metrics [52]–[59] to profile a vulnerability's own characteristics. How to assess the impact of a vulnerability in software supply chains is still an open problem.

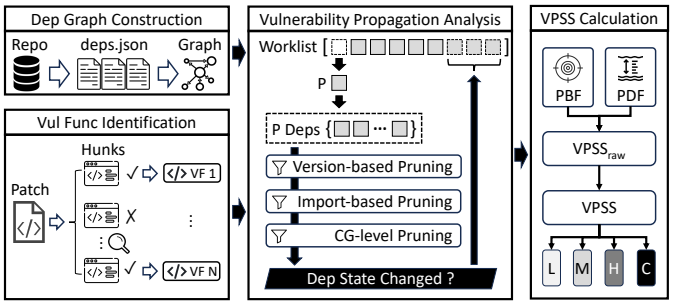


Fig. 1: Approach Overview

Overall, for vulnerability propagation analysis, an accurate and comprehensive solution should be a backward, transitive, CG-level analysis that considers complete dependency scope and project coverage and is capable of identifying VFs. Nevertheless, according to our comprehensive investigation, there still remains a gap between existing works and this goal. For vulnerability assessment, the community needs a new metric reflecting vulnerability impact in software supply chains.

III. Approach

A. Overview

Figure 1 illustrates the overview of our approach, which is designed to be ecosystem-agnostic and can be adapted to various programming languages by incorporating ecosystem-specific metadata formats and program analysis tools. This approach is composed of four steps:

Dependency Graph Construction (§ III-B). To carry out vulnerability propagation analysis, we first need to identify all

the downstream projects depending on the upstream project where the given vulnerability is located. We construct a *P*-level dependency graph for this purpose by analyzing all dependency declaration files from the target software ecosystem.

Vulnerable Function Identification (§ III-C). For CG-level analysis, VFs serve as the starting points. This step first generates a list of VF candidates using the patch-based method, and then takes a large language model (LLM)-assisted strategy to filter out vulnerability-irrelevant candidates.

Vulnerability Propagation Analysis (§ III-D). This step is to effectively identify all the downstream *PV*s in the whole ecosystem that directly or transitively call the VFs in the root vulnerable upstream *PV*s. We design a hierarchical worklist-based propagation algorithm to achieve this goal.

VPSS Calculation (§ III-E). This step calculates the VPSS score based on the results of the propagation analysis, which considers both the breadth and depth of the vulnerability impact scope in software supply chains.

B. Dependency Graph Construction

Given a vulnerability and the *PV*s affected by it, one of the preliminaries is to figure out which *PV*s depend on these vulnerable *PV*s. The graph structure can effectively organize *PV*s and their dependency information for querying. We call this directed graph a dependency graph. Based on the level of granularity, the dependency graph can be constructed in two distinct ways: the *PV*-level and the *P*-level design. In the *PV*-level dependency graph, nodes represent *PV*s and edges denote direct dependencies between them. In contrast, the *P*-level graph abstracts nodes as *P*s, with edges summarizing relations derived from the underlying *PV*-level dependencies.

Existing graph-based studies all construct *PV*-level dependency graphs. However, based on two observations, this option is not efficient for ecosystem-scale vulnerability propagation analysis. First, modern software ecosystems have become extremely large. For example, the Java Maven ecosystem has more than 15 million *PV*s [60], and the *PV*-level dependency graph could have tens of millions of nodes and even more edges. This huge scale makes the *PV*-level dependency graph too large to be queried efficiently, especially for transitive dependencies. Second, the number of *PV*s that depend on the upstream vulnerable *PV*s is usually a minority in a target ecosystem. Even among the *PV*s that do have dependencies, the ones that are actually affected are not many [14], which means that most of the nodes and edges in the *PV*-level dependency graph are irrelevant to the vulnerability propagation analysis. Therefore, it is not necessary and inefficient to construct a *PV*-level dependency graph for a whole ecosystem.

To address these issues, we propose to construct a *P*-level dependency graph. The number of *P*s in an ecosystem is usually much smaller than that of *PV*s, meaning that the *P*-level dependency graph has a much smaller scale than the *PV*-level dependency graph. The *P*-level graph can not only reduce computational overhead, but also serve as an efficient pre-filter, allowing queries to quickly narrow down the search space.

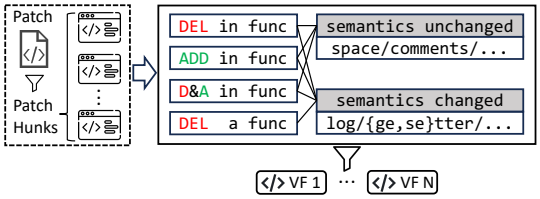


Fig. 2: Vulnerable Function Identification

With the initial traversal confined to a smaller, less complex *P*-level graph, the overall analysis becomes more scalable.

We follow a four-step procedure to build the *P*-level dependency graph. First, we download the ecosystem index and extract all the *PV* identifiers from the index into a list. Second, we obtain the dependency declaration files for *PV*s in this list from the official repository. Third, we parse the dependency declaration files to extract the dependencies of each *PV* and save them into `deps.json` files. Fourth, we construct the *P*-level dependency graph by analyzing the `deps.json` files of all the *PV*s. Specifically, for each *P*, we aggregate all the recorded dependent *PV*s from the `deps.json` files of the *PV*s that belong to it. It is important to note that we only build the dependency graph once and then continue to update it incrementally following the official repository, rather than building the entire dependency graph each time a new vulnerability is analyzed, which greatly reduces the time cost and improves the efficiency. During the propagation analysis (§ III-D) for a new vulnerability, a corresponding subgraph is queried, and we conduct a targeted *PV*-level inspection by querying the `deps.json` files when necessary, at which point the scale of the subject has been significantly reduced.

C. Vulnerable Function Identification

As presented in Figure 2, the VF identification step consists of two substeps: (1) patch-based VF candidate generation and (2) LLM-assisted VF filtering.

First, we parse the patch of the target vulnerability into individual hunks and only extract the function-modifying hunks. There are five types of function-modifying hunks: function *addition & deletion*, and internal *deletion & addition & modification (including deletion and addition)*. We filter out the function addition hunks as prior work [14] does because they are not the root cause of the vulnerability.

Second, we leverage LLMs for VF filtering. This method has three advantages: (1) LLMs possess broad domain knowledge across programming languages, enabling us to develop a language-agnostic and generalizable filtering approach. (2) LLMs are capable of recognizing non-standard syntax and syntactic sugar that are difficult to enumerate manually. (3) As LLMs continuously evolve and incorporate newly observed patterns from code corpora, their filtering capabilities remain up-to-date and adaptable, whereas manually maintained rules are often incomplete and costly to update.

Specifically, for the remaining VF hunk candidates, we design an in-context learning (ICL) [61] strategy and drive LLMs

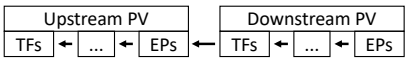
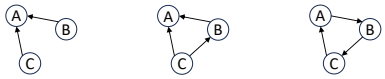


Fig. 3: Call Path Illustration



(a) (b) (c)

Fig. 4: Dependency Scenarios

to follow two filtering principles: semantics-equivalent modification and semantics-changing modification. If the semantics of a function remain equivalent after being modified by a hunk, the hunk is considered irrelevant to the vulnerability and should be filtered out, because it does not affect the existence of the vulnerability. For example, if a hunk only changes variable names, adds or deletes whitespaces, it is likely to be vulnerability-irrelevant. Even if a hunk changes the semantics of a function, it still can be irrelevant to the vulnerability. For example, if a hunk only adds or deletes logging or debugging code, it is likely to be irrelevant. To reduce incorrect filtering caused by LLMs, we also ask LLMs to provide reasons for decisions to conduct manual verification.

D. Vulnerability Propagation Analysis

This section describes the entire procedure of vulnerability propagation analysis. § III-D1 presents its overview that beginning with the root upstream *P*, each pass of the analysis handles the direct dependencies between the upstream *P* and its downstream *P*s. § III-D2 explains how the analysis prunes the downstream *PV*s with hierarchical methods to minimize the number of dependencies. Furthermore, § III-D3 shows the detailed algorithm workflow. Last, § III-D4 illustrates the entire procedure with a real-world example.

1) Overall Procedure: The vulnerability propagation analysis is to start from a root *P* (comprising a series of vulnerable *PV*s) with a vulnerability and identify all downstream *P*s (with the corresponding *PV*s) affected by this vulnerability at CG level along the dependency graph. Specifically, given an upstream *P*, we first query the dependency graph to get its direct downstream dependent *P*s, verify the validity of dependency between each pair of upstream and downstream *P*s by inspecting whether the downstream *PV*s transitively call the VFs in the root *PV*s, and then recursively propagate the analysis only for the truly affected downstream *P*s. Figure 3 illustrates this inter-*PV* analysis: For each *PV* of an upstream *P* in affected target versions (TVs), we first need to identify the entry-point functions (EPs) that can reach the target functions (TFs) in the same upstream *PV* and be called from outside. For *PV*s of the root *P*, VFs are the TFs. Then, we need to identify which downstream *PV*s call EPs in the upstream *PV*. If any, we need to record the functions that call upstream EPs in the downstream *PV*s, which serve as the TFs in future rounds.

In terms of vulnerability propagation, we need to consider three possible dependency scenarios. In Figure 4a, downstream B and C have individual dependencies on A. Therefore, we can identify affected downstream *PV*s for A, B, and C sequentially (`{A,B,C}`). In Figure 4b, downstream B and C share a common dependency on A, while C also has a dependency on B. In this scenario, analysis order `{A,B,C}` and `{A,C,B}` could potentially lead to different results. The result of `{A,C,B}` is potentially incomplete, because only EPs of upstream A exist when C is being processed, and the algorithm may miss the EPs of upstream B that are called by downstream C. For `{A,C,B}`, there should be a mechanism to ensure that C is analyzed again after its upstream EPs are updated (*i.e.,* B involves new EPs). In Figure 4c, the dependencies form a cycle, which could lead to infinite analysis loops if not handled properly. Although software dependencies are expected to form a directed acyclic graph [62], cyclic dependency relationships still exist in real-world software ecosystems. For example, *dom4j:dom4j:1.5.2* [63] and *jaxen:jaxen:1.1-beta-4* [64] have mutual dependencies on each other.

To effectively handle the aforesaid scenarios, we adopt the worklist algorithm, a well-established method in data-flow analysis frameworks, to systematically perform vulnerability propagation analysis over the dependency graph. Specifically, we maintain a worklist of *P*s whose states (*i.e.,* TVs, versioned reachable EPs, and affected downstream *PV*s) may still be updated. Initially, only the root *P* is added to the worklist. In each pass, we dequeue an item from the worklist, perform inter-*PV* analysis to update its downstream items, and enqueue the affected downstream *P*s if their associated versioned reachable upstream EPs have been updated. This approach ensures that each *P* is revisited only when necessary, effectively handling shared dependencies and cyclic structures while avoiding redundant analysis. The propagation continues until a fixed point is reached—when no new updates occur across the graph, ensuring both soundness and efficiency.

2) Pruning Mechanism: **Hierarchical Pruning.** We notice that there could be a large number of *PV* dependencies involved in the analysis, and the time and space costs associated with constructing CGs for large *PV*s are non-negligible. Consequently, it is computationally inefficient to analyze all dependencies directly at the CG level. To avoid lots of unnecessary fine-grained analysis, we employ a hierarchical pruning strategy, which first applies coarse-grained pruning methods to efficiently exclude false positive downstream dependent *PV*s (as well as their corresponding *P*s, if all associated *PV*s are pruned out), and then performs the fine-grained CG-level analysis only on the remaining downstream candidates. As shown in the middle part of Figure 1, this hierarchical pruning mechanism comprises three levels of pruning: *Version-based pruning* excludes downstream *PV*s that do not declare a dependency on the specific upstream TVs. *Import-based pruning* further eliminates downstream *PV*s that, despite declaring a dependency, do not actually import or include upstream contents. *CG-level pruning* finally removes downstream *PV*s

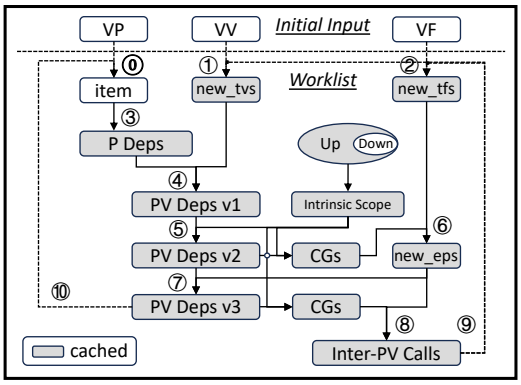


Fig. 5: Vulnerability Propagation Analysis

Algorithm 1: Worklist-based Propagation Algorithm

Input: Vulnerable P $\mathcal{VP}$, vulnerable versions $\mathcal{VV}$, vulnerable functions $\mathcal{VF}$
Output: All the cached analysis results in Figure 5

```
1  ROOT ← true, worklist ← {VP}
2  while worklist is not empty do
3      ⓪ item ← worklist.pop()
4      (tvs, tfs) ← getTVAndTF(item, VV, VF, ROOT)
5      ROOT ← false
6      (old_tvs, old_tfs) ← loadOldTVAndTF(item)
7      ① new_tvs ← diffMergeSave(old_tvs, tvs)
8      ② new_tfs ← diffMergeSave(old_tfs, tfs)
9      ③ pdeps ← genPDeps(item)
10     // hierarchical pruning: ④ – ⑧
11     (s, v3) ← prune(item, pdeps, new_tvs, new_tfs)
12     if s is changed then
13         ⑨ propagateTVAndTF(item, v3)
14         ⑩ worklist.extend(v3)
```

that do not invoke any of the upstream EPs at the CG level.

Handling Fat Packages. An issue in the pruning mechanism arises from the presence of fat *PV*s—release packages that bundle not only a project's own code but also its dependencies. Such packaging practices, common in ecosystems like Java where fat JARs are widely used, can interfere with precise analysis by conflating intrinsic and extrinsic program elements. For efficient analysis, only the *intrinsic scope* (*i.e.,* the program components that are native to the project itself) should be considered. To address this issue, we propose a general method for identifying the intrinsic scope of a given *PV*, even when fat packaging practices differ across ecosystems. Specifically, for a target *PV*, we first obtain the set of files in its release package, denoted as *Up*. We then collect the release files of all its declared dependencies as set *Down*. By subtracting *Down* from *Up*, we obtain the intrinsic scope of the *PV*, which serves as the foundation for import-based and CG-level pruning.

3) Algorithm Workflow: Taking all the above into consideration, we design a hierarchical worklist-based algorithm to perform vulnerability propagation analysis, presented in Figure 5, with the corresponding steps annotated using circled numbers that match those in the pseudocode provided in Algorithm 1 for clarity. Given a vulnerability, the algorithm takes three inputs: the root *P* of the vulnerability ($\mathcal{VP}$), the vulnerable versions of the root *P* ($\mathcal{VV}$), and the vulnerable functions of the root *P* ($\mathcal{VF}$). The worklist is initialized with the root *P*, and the algorithm iteratively processes items in it.

At the beginning of each pass (step ⓪), one item (*P*) is fetched from the worklist. At step ①, the algorithm generates the TV list (new_tvs) of current item that are affected by the vulnerability. Specifically, if it is the first pass, the list is initialized with $\mathcal{VV}$. Otherwise, the current item must have its own upstream *P*s, the list of which is saved and updated in the previous passes (step ⑨). Consequently, the algorithm queries the *Inter-PV Calls* record of each upstream *P* in this list to get the latest affected versions of the current item into tvs. It then loads the cached old version list (old_tvs) if the current *P* has been processed in previous passes, and derives the difference between the two lists to get the new affected versions (new_tvs). The combination of old_tvs and tvs is cached for future use. At step ②, the algorithm generates the TF list (new_tfs) of the current item that are affected by the vulnerability. The generation process is similar to the generation of new_tvs. Also, the combination of old_tfs and tfs is cached for future use. Then, at step ③, the algorithm queries the dependency graph to extract the dependency relationships between the current item and its direct downstream dependent *P*s into $pdeps$.

With $pdeps$, new_tvs, and new_tfs available, the hierarchical pruning begins. At step ④, the algorithm generates the *PV* dependencies for the current upstream *P* by querying the `deps.json` records belonging to *P*s in $pdeps$, and then prunes out irrelevant *PV* dependencies by checking whether the downstream *PV*s rely on upstream *PV*s covered by new_tvs. After this step, the dependency relationships between upstream *PV*s and the remaining downstream *PV*s are used to generate or update *PV Deps v1*. At step ⑤, the algorithm further prunes dependencies by checking whether the downstream *PV*s import contents from upstream *PV*s in *PV Deps v1*. The check is restricted to the intrinsic scope for both upstream and downstream *PV*s. After this step, *PV Deps v2* is generated or updated. Then, at step ⑥, the algorithm identifies new EPs (new_eps) for CG-level pruning. To achieve this, the algorithm loads the cached old EP list, derives an EP list (eps) by constructing CG and conducting backward BFS traversal from new_tfs to externally accessible functions, and uses the difference between the two lists as new_eps for each upstream *PV*. The combination of old_eps and eps is cached for future use. Notably, empty new_eps for all the upstream *PV*s indicates that the dependency state s of the current item has not changed, and the algorithm will not append its downstream *P*s to the worklist. With new_eps available, at step ⑦, the algorithm prunes the dependencies by checking whether downstream *PV*s call EPs in new_eps. After this step, *PV Deps v3* ($v3$ in Algorithm 1 for brevity) is

generated or updated. At the end of the pruning process, the algorithm generates and caches the *Inter-PV Calls* record for the current item at step ⑧.

At step ⑨, the algorithm sends the inter-*PV* information (versions and calls) to each downstream *P* from *PV Deps v3* for future passes. Finally, at step ⑩, the algorithm appends the downstream *P*s from *PV Deps v3* to the worklist. The algorithm continues until the worklist becomes empty.

4) Example: We illustrate the algorithm workflow process using CVE-2016-5393 [65], a high-severity vulnerability in *org.apache.hadoop:hadoop-common*. In affected versions of Hadoop, a remote attacker can execute commands with HDFS privileges. For brevity, we present only the first round of propagation analysis. Further propagation results and impact quantification of CVE-2016-5393 are presented in § V-D.

At step ⓪, *org.apache.hadoop:hadoop-common* is popped as the root *P*. At steps ①–②, its vulnerable versions and functions are set as targets. At step ③, the algorithm queries the dependency graph and finds 1,543 downstream *P*s. At step ④, version-based pruning removes dependents on non-vulnerable *PV*s; for example, *com.wgzhao.addax:hbase20xreader:4.0.3* depends on a non-vulnerable version and is pruned, leaving 491 *P*s and 4,258 *PV*s as *PV Deps v1*. At step ⑤, import-based pruning excludes dependents that declare but never import upstream classes; for example, although *com.fiftyonezero.eel:eel-elasticsearch_2.10:0.11.0* claims that it depends on a vulnerable version of upstream *PV*, it actually does not import any class from this *PV* and is pruned out. After the pruning, *PV Deps v2* is generated, which contains 373 *P*s and their 3,321 corresponding *PV*s. At step ⑥–⑦, CG-level pruning removes dependents that import but do not call vulnerable entrypoints; for instance, although *com.tencent.angel:angel-ps-graph:2.4.0* depends and imports classes from a vulnerable upstream *PV*, it actually does not call any entrypoint from this *PV* and is pruned out here. After the pruning, *PV Deps v3* is generated, which contains 228 *P*s and their 1,685 corresponding *PV*s. Finally, steps ⑧–⑩ update the inter-*PV* information and enqueue downstream *P*s in *PV Deps v3* for the next round.

E. VPSS Calculation

After the propagation analysis, we obtain graph-based statistics that reflect a vulnerability's impact across the ecosystem. However, these raw data are not readily interpretable or actionable. To better profile this impact, we introduce the Vulnerability Propagation Scoring System (VPSS)—a metric that transforms propagation data into a standardized and meaningful impact score for software supply chain vulnerabilities.

VPSS is defined as a propagation-aware measure of vulnerability impact that combines two dimensions: the breadth (the affected share of downstream packages) and the depth (the length of dependency chains). Higher scores therefore directly reflect wider and deeper propagation in the ecosystem. The design of VPSS follows three key principles: (1) Graph awareness: The metric should incorporate both breadth and depth in the dependency graph to capture how widely and deeply a vulnerability propagates. (2) Interpretability and compatibility: VPSS must be easy to understand, ensuring seamless integration into current vulnerability management workflows and complementing static severity metrics such as CVSS. (3) Time-awareness: As ecosystems evolve, the metric should adapt to changes such as new dependencies or patches. VPSS is thus a dynamic score, supporting longitudinal tracking and timely risk assessment.

To apply these principles, VPSS transforms the results of vulnerability propagation analysis into a normalized impact score within the 0–10 range. As shown in Figure 1, the score is divided into four tiers—*low* (0–4), *medium* (4–7), *high* (7–9), and *critical* (9–10)—for intuitive risk interpretation.

VPSS captures the breadth and depth of vulnerability propagation through two multiplicative factors: *Propagation Breadth Factor* (PBF), which quantifies how widely a vulnerability spreads via direct and transitive downstream dependencies, and *Propagation Depth Factor* (PDF), which measures how deeply it penetrates the dependency graph based on propagation chain length. These two factors define the raw score:

$$VPSS_{\text{raw}} = PBF \times PDF \tag{1}$$

The PBF component is computed from four normalized ratios representing the proportion of affected downstream *P* and *PV* entities, separately for direct and transitive dependencies. Here, Total_P and Total_PV denote the total number of *P*s and *PV*s in the target ecosystem, respectively, which are obtained in the construction process of the dependency graph.

$$r_{\text{p_dir}} = \frac{P_{\text{dir}}}{\text{Total_P}}, \quad r_{\text{p_trans}} = \frac{P_{\text{trans}}}{\text{Total_P}},$$
$$r_{\text{pv_dir}} = \frac{PV_{\text{dir}}}{\text{Total_PV}}, \quad r_{\text{pv_trans}} = \frac{PV_{\text{trans}}}{\text{Total_PV}}$$

These values are aggregated using a weighted sum. Generally, the relationship between these weights should be $w_{\text{p_dir}} > w_{\text{pv_dir}} > w_{\text{p_trans}} > w_{\text{pv_trans}}$:

$$W = \begin{pmatrix} w_{\text{p_dir}} & w_{\text{p_trans}} & w_{\text{pv_dir}} & w_{\text{pv_trans}} \end{pmatrix}$$
$$X = \begin{pmatrix} r_{\text{p_dir}} & r_{\text{p_trans}} & r_{\text{pv_dir}} & r_{\text{pv_trans}} \end{pmatrix}$$

To avoid concentration of PBF values in a narrow range, we apply a logarithmic scaling with an amplification factor γ:

$$PBF = \ln\left(1 + \gamma \cdot WX^{\top}\right) \tag{2}$$

The PDF component is more straightforward, measuring the average and maximum depth of propagation paths on the dependency graph, where L_{norm} is a normalization constant used to adjust the depth metric to a reasonable scale:

$$PDF = 1 + \frac{L_{\max} + L_{\text{avg}}}{2L_{\text{norm}}} \tag{3}$$

The raw VPSS score is then normalized to the final 0–10 range using an exponential saturation function, where k is the saturation parameter controlling the rate at which the raw scores are converted to the final scores:

$$VPSS = 10 \times \left(1 - \exp\left(-\frac{VPSS_{\text{raw}}}{k}\right)\right) \tag{4}$$

In total, VPSS introduces seven parameters—$w_{\text{p_dir}}$, $w_{\text{p_trans}}$, $w_{\text{pv_dir}}$, $w_{\text{pv_trans}}$, γ, L_{norm}, and k—whose values affect the scaling and sensitivity of the score. Currently, we set these parameters based on domain knowledge and empirical tuning. We leave automating their setting based on statistical learning from historical vulnerability data as future work.

Lastly, to reflect the evolving nature of software ecosystems, VPSS is explicitly time-aware. As new software versions are released and patches are applied, the downstream impact of a vulnerability naturally diminishes. Particularly, when calculating a time-aware VPSS score at t, all *P*s and *PV*s released later than t will be excluded. Therefore, each VPSS score corresponds to a specific snapshot in time.

Rationality Analysis. The rationality of VPSS lies in its ability to capture the essential characteristics of vulnerability propagation in software supply chains. By combining breadth and depth, the metric reflects that ecosystem risk grows disproportionately when vulnerabilities spread both widely and deeply, whereas either factor alone provides only a partial view. The weighting scheme emphasizes direct dependencies and project-level entities, because (1) direct dependencies face greater risks than transitive dependencies due to the different chances of exploitability [14], and (2) *P* dependencies are more stable and reflect their impact on the entire ecosystem, while *PV* dependencies may be less stable due to version fluctuations. Our empirical evaluation (§ V-C and § V-D) shows that high VPSS values correspond to vulnerabilities with wide and deep propagation (*e.g.,* CVE-2016-5393), while vulnerabilities with limited downstream use receive lower scores. Moreover, the temporal evolution of VPSS naturally reflects the adoption of patched versions, demonstrating that the metric captures realistic dynamics of vulnerability impact. These observations demonstrate that VPSS provides a principled and realistic measure of supply chain vulnerability impact.

IV. Implementation

We implement an approach prototype for the Java Maven ecosystem in 2.3K lines of Python and 1K lines of Java code. In this section, we present the implementation details.

Dependency Graph Construction. We download the Maven repository index from the Maven Central Repository (MCR) [28], and parse it with Apache Lucene to extract the *PV* information. Given the variability of version conflict resolution across projects (*e.g.,* dependency order, exclusions, and overrides), to ensure scalable analysis without requiring build-specific contexts, we choose not to simulate Maven's full resolution logic, but to use Maven Model Builder to extract the dependency information from the POM files. We follow prior work [9] to filter out non-deployed dependencies whose `scope` are not `compile` or `runtime`. Finally, we build the *P*-level dependency graph with NetworkX, and store it in a Neo4j graph database for efficient querying.

Vulnerable Function Identification. For the filtering process, we use the GPT-4o-mini API provided by OpenAI to filter out the vulnerability-irrelevant VF candidates. We also test other mainstream LLMs (*e.g.,* Gemini 2.5 Flash and Claude Sonnet 4) as well as a locally deployed open-source model (Qwen2.5-Coder-32B-Instruct). Their results are comparable, indicating that our approach benefits from the general reasoning and semantic understanding capabilities of modern LLMs rather than any specific proprietary service. Since the interface to different models is unified, substituting one for another does not affect the overall workflow.

Vulnerability Propagation Analysis. We query the dependency graph stored in the Neo4j database to obtain all the downstream dependencies of specific *P*. To verify whether a downstream *PV* imports an upstream *PV*, we use Java Dependency Analysis Tool to analyze the JAR files. To quickly determine whether an upstream method is called by in a downstream *PV*, we utilize Java ASM to parse and search in the bytecode of JAR files. For CG-level analysis, we use Soot [66] to analyze JAR files and generate CGs.

V. Evaluation

In this section, we evaluate the effectiveness of our approach by answering the following research questions (RQs):

RQ1: How effective and scalable is our ecosystem-scale vulnerability propagation analysis in identifying potentially affected downstream software projects? (§ V-B)

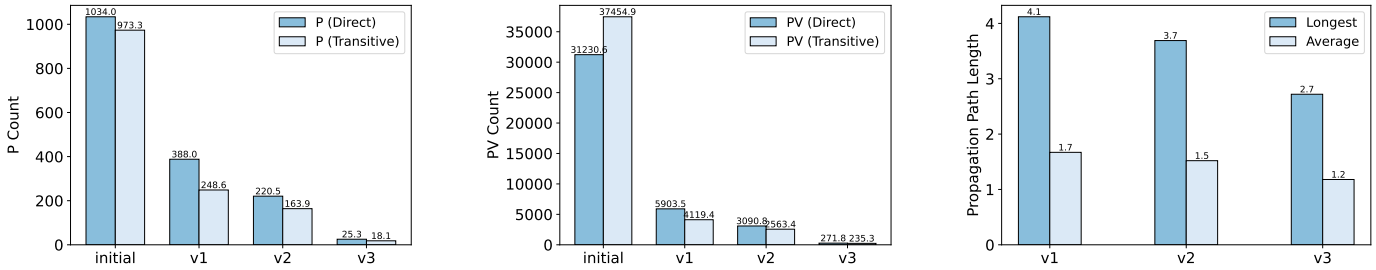
RQ2: What statistical insights can be drawn from the VPSS scores computed across real-world vulnerabilities? (§ V-C)

All the experiments are conducted on a server equipped with an AMD EPYC 9184X 16-Core Processor and 500 GB of physical memory, running Ubuntu 22.04.5 LTS on the host.

A. Dataset Preparation

To evaluate the effectiveness of our full-ecosystem vulnerability propagation analysis, we build upon the dataset released by Wu *et al.* [14], which contains over 800 vulnerabilities in Maven. However, running our full analysis on all 800+ vulnerabilities would impose a heavy burden on MCR and significantly increase computational cost. To balance evaluation thoroughness and practical feasibility, we randomly sample 100 vulnerabilities as our experimental dataset.

With this dataset, we make the following enhancements. First, in Wu *et al.*'s dataset, only one vulnerable version was selected for each CVE, which can not fully capture the affected version range. To address this limitation, we enhance the dataset by incorporating the complete list of vulnerable versions for each CVE from the National Vulnerability Database (NVD) [67]. Second, we identify inaccurate VF annotations in the original dataset with our method in § III-C, and remove two incorrect VFs. Figure 7 shows excerpts from the corresponding patches [68], [69]. In CVE-2021-43795 [70], the only change is replacing double quotes with single quotes around the character '?', without altering statement semantics. Thus, the `toString()` method should not be considered a VF. In CVE-2021-26118 [71], the removed *Setter* and *Getter* methods for `AMQSession` only access `this.advisorySession` and contain no vulnerability-related logic. In summary, each entry in our refined dataset contains the groupId and artifactId of the vulnerable project, an augmented list of vulnerable versions, and a set of vulnerable functions.



(a) Average P (Direct vs Transitive) (b) Average PV (Direct vs Transitive) (c) Longest and Average Path Length

Fig. 6: Average propagation statistics across pruning stages. The v1, v2, and v3 are associated with the pruning results *PV Deps v1*, *PV Deps v2*, and *PV Deps v3* in § III-D, respectively.

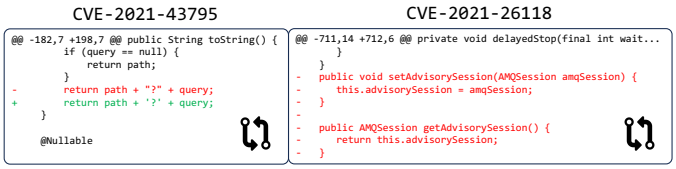


Fig. 7: Excerpts of Patches for CVE-2021-{43795,26118}

B. Vulnerability Propagation Analysis

For each vulnerability in the dataset, we run our prototype to assess its impact in the Maven ecosystem. We use the snapshot of MCR index on December 26, 2024, to construct the dependency graph. There are around 660K *P*s in the dependency graph, much fewer than the 15M *PV*s in MCR. During the analyzing process, we limit the request frequency when downloading JAR files to avoid excessive load on MCR.

Excluding the time spent on downloading the JAR packages, the per-vulnerability analysis time ranges from 1.2 seconds to 54 hours, with an average of 5.2 hours and a median of 1.5 hours. Considering the ecosystem-wide scope of our analysis, this overhead is acceptable and demonstrates the practicality of our approach for large-scale vulnerability impact assessment. Moreover, in our measurement each vulnerability was analyzed independently from scratch, whereas in practice, intermediate artifacts such as CGs or intrinsic scope information of *PV*s can be reused across vulnerabilities, which will make future analyses even more efficient.

To further evaluate the effectiveness of our hierarchical worklist-based propagation algorithm, we collect data on the number of *P*s and *PV*s involved in vulnerability propagation at different stages of the algorithm, as well as the length of the longest propagation paths and the average length of propagation paths on the dependency graph. Figure 6 presents the average values of these data at different stages. For direct and transitive *P* (Figure 6a) and *PV* dependencies (Figure 6b), we obtain the average values before the whole pruning process and after each pruning step. For dependency paths (Figure 6c), we find that getting the average and longest path length before pruning would take too long to compute, so we decide to only look at the results after each pruning step.

We obtain the following findings from the analysis results: First, the hierarchical pruning mechanism is quite effective, as 97.8% *P*s and 99.2% *PV*s on average are pruned out during the vulnerability propagation analysis. Also, the length of the longest path and average length decrease at least by 34.1% and 29.4%, respectively. In addition, all the statistics in Figure 6 decrease in stages, confirming that every pruning process makes its own contributions. Considering the time and space cost of constructing CGs, our approach greatly reduces the number of CGs that need to be built. Second, performing vulnerability propagation analysis at the *PV* level based only on project-declared dependencies will result in a large number of false positives, as 94.9% *PV*s are pruned out with import-based pruning and CG-level pruning. As shown above, pruning dramatically reduces the number of downstream candidates, which directly translates into fewer CG constructions and proportionally lower analysis time. In contrast, a fully non-pruning baseline would require constructing CGs for essentially all downstream packages in the Maven ecosystem, which is computationally infeasible.

C. VPSS Statistics

After the propagation analysis is completed for the dataset, we collect results for VPSS calculation. As mentioned in § III-E, we determine the parameter values based on preliminary experiments and expert knowledge, aiming to balance the influence of different components and ensure that VPSS scores meaningfully reflect the propagation impact—higher scores correspond to wider and deeper impact in supply chains. Particularly, we empirically set the parameters for VPSS computation as follows: $w_{\text{p_dir}} = 5$, $w_{\text{p_trans}} = 2.5$, $w_{\text{pv_dir}} = 3$, and $w_{\text{pv_trans}} = 1.5$, $\gamma = 500$, $L_{\text{norm}} = 10$, and $k = 0.5$.

With such settings, for each vulnerability, we sample 24 time points at 30-day intervals starting from its disclosure date in NVD (denoted as t_0). We compute the VPSS score at each point (t_0 to t_{23}) to capture the evolution of the vulnerability's impact on software supply chains over approximately 24 months. To evaluate whether VPSS effectively captures the temporal and distributional characteristics of vulnerability propagation, we visualize its evolution and overall trends. Specifically, we present Figure 8 to highlight VPSS trajectories of the top-10 most impactful CVEs, and Figure 9 to show the distribution of VPSS scores across all 100 CVEs over time.

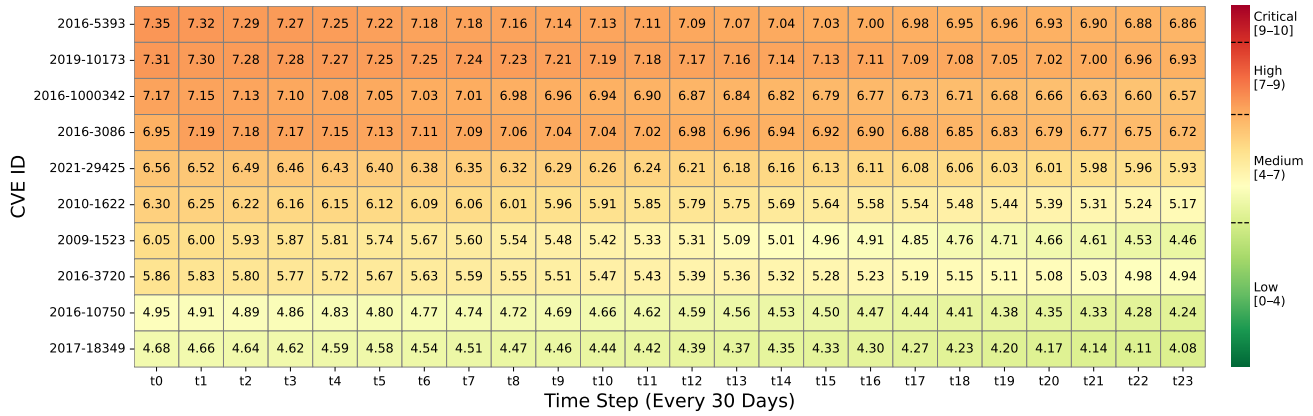


Fig. 8: VPSS Time Series (Every 30 Days) for Top-10 CVEs (t0 to t23)

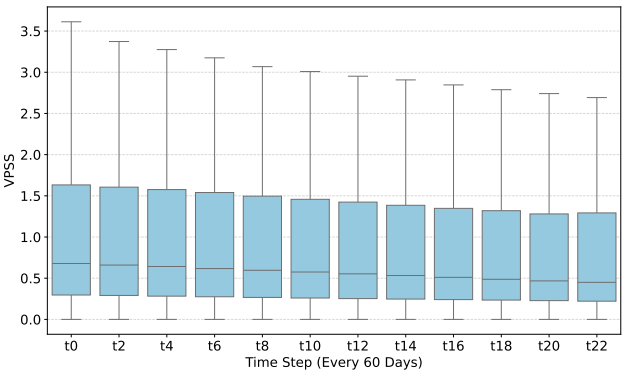


Fig. 9: VPSS Distribution (Every 60 Days) Across 100 CVEs

As shown in Figure 8, at t_0, the top-10 VPSS scores scale from 7.35 to 4.68, reflecting risk levels from *high* to *medium*. Nevertheless, the VPSS scores generally decline over time, indicating a decreasing impact on software supply chains. This trend can be attributed to two main factors: (1) as patched versions are released, an increasing number of downstream projects migrate to vulnerability-free versions; and (2) over time, more projects emerge in the Maven ecosystem. Additionally, we observe an unusual increase in the VPSS score of CVE-2016-3086 [29] from t_0 to t_1 (from 6.95 to 7.19). This anomaly may be explained by delayed dependency updates in downstream projects [14]. Lastly, under the current parameter settings, none of the vulnerabilities in our dataset reach the *critical* VPSS risk level, leaving room for potentially higher-impact vulnerabilities beyond the scope of the current dataset.

Similar to the top-10 CVEs, in Figure 9, the distribution of VPSS scores across all 100 CVEs shows a gradual decline in both median and interquartile range over time. The boxplots reveal that the majority in the dataset maintain relatively low VPSS scores. Over time, the overall dispersion narrows slightly, suggesting that the propagation effects of most vulnerabilities tend to stabilize or diminish within two years. This aligns with the expected lifecycle of patch adoption and ecosystem decoupling from vulnerable packages.

D. Case Study: CVE-2016-5393

This example (CVE-2016-5393 [65]) shows how our framework captures vulnerability propagation in software supply chains. Our analysis shows that it exhibits substantial propagation at the ecosystem level. At t_0, the vulnerability directly affected 228 *P*s and transitively propagated to 154 others, impacting a total of 618 direct and 321 transitive *PV*s. The resulting VPSS score reached 7.35, which is the highest among all CVEs in our dataset.

Over 24 months, its VPSS score exhibited a gradual decline to 6.86, indicating a slow mitigation pace, which reflects long-tail dependency retention in real-world ecosystems. Notably, the longest propagation chain consists of 7 dependency hops, spanning critical components of the Hadoop and Hive data processing stacks, illustrating how a single low-level vulnerability can affect a wide range of downstream components. The average path length also increased slightly over time, reaching 2.33 by t_{23}, indicating deepening propagation.

VI. Discussion

A. Accuracy of Propagation Analysis

There are three main factors that affect the accuracy of our vulnerability propagation analysis. First, vulnerable versions from public databases such as NVD may be incomplete or inaccurate [72], leading to false positives or negatives in propagation results. This can be mitigated by version identification methods [73]–[75]. Second, although we enhance patch-based VF identification, real-world security patches occasionally include unrelated but substantial code changes, complicating accurate VF extraction. We leave refining VF identification as future work [76]. Third, to achieve ecosystem-scale analysis, we rely on static techniques, *e.g.,* import analysis and CG construction. While efficient, static analysis may miss paths involving dynamic features (*e.g.,* class loading or method dispatching), causing false negatives. Techniques on reflection resolution [77], [78] can be integrated into the *import-based pruning* and *CG-level pruning* stages to improve the accuracy of class dependency analysis and CG construction.

B. Adaptability of the Proposed Approach

While our implementation targets the Java Maven ecosystem, the approach is broadly applicable: (1) Some Java projects may exist outside of the Maven ecosystem (*e.g.,* hosted only on GitHub). While we follow prior large-scale studies with Maven [13], [14], our approach does not rely on Maven-specific assumptions: as long as an index of external Java projects can be constructed with dependency metadata, our framework can seamlessly incorporate them into the analysis. (2) Our framework is ecosystem-agnostic, which can be instantiated in other ecosystems. For example, in the Python ecosystem, a dependency graph can be built by parsing `setup.py`, `requirements.txt`, or `pyproject.toml`, import relations can be extracted using tools such as pydeps [79], and CGs can be generated via static analyzers like PyCG [80].

C. Parameter Setting of VPSS

For VPSS framework (§ III-E), we introduce a set of configurable parameters to enhance its flexibility across different analytical contexts. In evaluation, we instantiate these parameters with fixed values based on domain expertise (§ V-C). While this manual configuration suffices for our work, a promising direction for future work is to explore data-driven approaches for parameter tuning. For example, one could employ statistical optimization or learn optimal settings from historical vulnerability propagation data, enabling more adaptive and context-aware scoring across diverse software ecosystems.

D. Application of VPSS

The VPSS framework offers three key applications. First, after a vulnerability is disclosed, VPSS enables quantification of its impact across software supply chains. It can be used with CVSS to provide more actionable and early-warning signals. Second, VPSS can be integrated into vulnerability management workflows to enhance the prioritization process, ensuring that remediation efforts focus on weaknesses with the greatest propagation risk. Finally, by translating the complex software interdependencies into a standardized score, VPSS facilitates the quantification of software supply chain risk for cyber-insurance underwriting, supporting more granular and data-driven policy design and premium calculation.

VII. Related Work

A. Vulnerability Propagation Analysis

Existing work on vulnerability propagation spans Java, JavaScript, and Python ecosystems: In Java, Wu *et al.* [14] and Mir *et al.* [15] perform CG-level reachability analyses on global and subset Maven graphs; Ponta *et al.* [5], [8], [9], [12], [33] combine static and dynamic methods for application-level VF detection; Zhang *et al.* [17] determine whether a given project is threatened by vulnerabilities by establishing and querying a vulnerable API database; others parse POM files for one-hop dependency analyses [4], [6], [7] or build dependency graphs for direct and transitive analyses [10], [11], [13], [16], [18]. In JavaScript, empirical studies trace client-side library usage and vulnerability inclusions [19], npm direct dependencies [20], and ecosystem-wide propagation via dependency graphs [21]–[23]. In Python, Ma *et al.* [24] propose a two-stage impact estimation for scientific projects.

B. Library Vulnerability Exploitation

Beyond propagation analysis, researchers propose methods to generate exploits for library vulnerabilities: Iannone *et al.* [81] employ genetic algorithms to evolve test cases that start from clients and reach vulnerable sites in libraries. Kang *et al.* [82] propose to execute vulnerability-revealing tests from libraries to capture triggering states, and guides evolutionary test generation in clients to reproduce these state. Zhou *et al.* [83] combine LLM-guided seed generation with directed fuzzing along call chains to incrementally exploit library vulnerabilities. Chen *et al.* [84] use exploits to guide test generation by migrating crafted parameters into project tests.

C. Vulnerability Assessment

Researchers have proposed several approaches to improving existing assessments and conducting novel assessments. Among them, some works aim to automatically predict the CVSS scores [41]–[46]; some propose to automate the Common Weakness Enumeration classification task [47]–[51]. Additionally, to better understand and profile vulnerabilities, researchers have begun to study more characteristics of them [85]. One active area is exploitation prediction [52]–[59], which adopts data-driven techniques to estimate the likelihood that a vulnerability will be exploited in the wild.

VIII. Conclusion

This paper fills two key gaps in software supply chain security: the lack of accurate whole-ecosystem vulnerability propagation analysis and the absence of quantitative indicators for assessing vulnerability propagation impact. We propose a novel framework that combines a hierarchical worklist-based algorithm with multi-level pruning to enable scalable, call-graph-level propagation analysis across direct and transitive dependencies. We introduce the *Vulnerability Propagation Scoring System* (VPSS), a graph-based metric capturing both propagation breadth and depth over time. Evaluations on Java Maven ecosystem and real-world CVEs demonstrate the effectiveness of our approach in assessing supply chain risk.

Acknowledgment

We thank Zhenxi Li, Shaofei Li, and Yuancheng Jiang for their assistance and the anonymous reviewers for their valuable comments. This research is supported by the National Research Foundation, Singapore, through the National Cybersecurity R&D Lab at the National University of Singapore under its National Cybersecurity R&D Programme (Award No. NCR25-NCL P3-0001). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore, and National Cybersecurity R&D Lab at the National University of Singapore.

References

[1] C. Zhang, J. Zeng, Y. Zhang, A. Ahmad, F. Zhang, H. Jin, and Z. Liang, "The hitchhiker's guide to high-assurance system observability protection with efficient permission switches," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 3898–3912.

[2] MITRE, "CVE - CVE-2014-0160," https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2014-0160, 2014.

[3] CSA, "The Log4Shell Vulnerability," https://www.csa.gov.sg/resources/publications/the-log4shell-vulnerability, 2022.

[4] M. Cadariu, E. Bouwers, J. Visser, and A. Van Deursen, "Tracking known security vulnerabilities in proprietary software systems," in *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 2015, pp. 516–519.

[5] H. Plate, S. E. Ponta, and A. Sabetta, "Impact assessment for vulnerabilities in open-source software libraries," in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2015, pp. 411–420.

[6] R. G. Kula, D. M. German, A. Ouni, T. Ishio, and K. Inoue, "Do developers update their library dependencies? an empirical study on the impact of security advisories on library migration," *Empirical Software Engineering*, vol. 23, pp. 384–417, 2018.

[7] D. Du, X. Ren, Y. Wu, J. Chen, W. Ye, J. Sun, X. Xi, Q. Gao, and S. Zhang, "Refining traceability links between vulnerability and software component in a vulnerability knowledge graph," in *Web Engineering: 18th International Conference, ICWE 2018, Cáceres, Spain, June 5-8, 2018, Proceedings 18*. Springer, 2018, pp. 33–49.

[8] S. E. Ponta, H. Plate, and A. Sabetta, "Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software," in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2018, pp. 449–460.

[9] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, and F. Massacci, "Vulnerable open source dependencies: Counting those that matter," in *Proceedings of the 12th ACM/IEEE international symposium on empirical software engineering and measurement*, 2018, pp. 1–10.

[10] W. Hu, Y. Wang, X. Liu, J. Sun, Q. Gao, and Y. Huang, "Open source software vulnerability propagation analysis algorithm based on knowledge graph," in *2019 IEEE International Conference on Smart Cloud (SmartCloud)*. IEEE, 2019, pp. 121–127.

[11] Y. Wang, B. Chen, K. Huang, B. Shi, C. Xu, X. Peng, Y. Wu, and Y. Liu, "An empirical study of usages, updates and risks of third-party libraries in java projects," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 35–45.

[12] S. E. Ponta, H. Plate, and A. Sabetta, "Detection, assessment and mitigation of vulnerabilities in open source dependencies," *Empirical Software Engineering*, vol. 25, no. 5, pp. 3175–3215, 2020.

[13] L. Zhang, C. Liu, S. Chen, Z. Xu, L. Fan, L. Zhao, Y. Zhang, and Y. Liu, "Mitigating persistence of open-source vulnerabilities in maven ecosystem," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 191–203.

[14] Y. Wu, Z. Yu, M. Wen, Q. Li, D. Zou, and H. Jin, "Understanding the threats of upstream vulnerabilities to downstream projects in the maven ecosystem," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1046–1058.

[15] A. M. Mir, M. Keshani, and S. Proksch, "On the effect of transitivity and granularity on vulnerability propagation in the maven ecosystem," in *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2023, pp. 201–211.

[16] Z. Ma, S. Mondal, T.-H. Chen, H. Zhang, and A. E. Hassan, "Vulnet: Towards improving vulnerability management in the maven ecosystem," *Empirical Software Engineering*, vol. 29, no. 4, p. 83, 2024.

[17] F. Zhang, L. Fan, S. Chen, M. Cai, S. Xu, and L. Zhao, "Does the vulnerability threaten our projects? automated vulnerable api detection for third-party libraries," *IEEE Transactions on Software Engineering*, 2024.

[18] Y. Shen, X. Gao, H. Sun, and Y. Guo, "Understanding vulnerabilities in software supply chains," *Empirical Software Engineering*, vol. 30, no. 1, pp. 1–38, 2025.

[19] T. Lauinger, A. Chaabane, S. Arshad, W. Robertson, C. Wilson, and E. Kirda, "Thou shalt not depend on me: Analysing the use of outdated javascript libraries on the web," *arXiv preprint arXiv:1811.00918*, 2018.

[20] A. Decan, T. Mens, and E. Constantinou, "On the impact of security vulnerabilities in the npm package dependency network," in *Proceedings of the 15th international conference on mining software repositories*, 2018, pp. 181–191.

[21] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *28th USENIX Security symposium (USENIX security 19)*, 2019, pp. 995–1010.

[22] Y. Wang, P. Sun, L. Pei, Y. Yu, C. Xu, S.-C. Cheung, H. Yu, and Z. Zhu, "Plumber: Boosting the propagation of vulnerability fixes in the npm ecosystem," *IEEE Transactions on Software Engineering*, vol. 49, no. 5, pp. 3155–3181, 2023.

[23] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, and X. Peng, "Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 672–684.

[24] W. Ma, L. Chen, X. Zhang, Y. Feng, Z. Xu, Z. Chen, Y. Zhou, and B. Xu, "Impact analysis of cross-project bugs on software ecosystems," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 100–111.

[25] FiRST, "CVSS v4.0 FAQ," https://www.first.org/cvss/v4.0/faq, 2025.

[26] U. Khedker, A. Sanyal, and B. Sathe, *Data flow analysis: theory and practice*. CRC Press, 2017.

[27] Y. Jiang, C. Zhang, B. Ruan, J. Liu, M. Rigger, R. H. Yap, and Z. Liang, "Fuzzing the {PHP} interpreter via dataflow fusion," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 6143–6158.

[28] H. Boutemy, "Central Index," https://maven.apache.org/repository/central-index.html, 2025.

[29] NVD, "NVD - CVE-2016-3086," https://nvd.nist.gov/vuln/detail/CVE-2016-3086, 2017.

[30] J. Dai, Y. Zhang, Z. Jiang, Y. Zhou, J. Chen, X. Xing, X. Zhang, X. Tan, M. Yang, and Z. Yang, "{BScout}: Direct whole patch presence test for java executables," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 1147–1164.

[31] Z. Jiang, Y. Zhang, J. Xu, Q. Wen, Z. Wang, X. Zhang, X. Xing, M. Yang, and Z. Yang, "Pdiff: Semantic-based patch presence testing for downstream kernels," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1149–1163.

[32] B. Liu, G. Meng, W. Zou, Q. Gong, F. Li, M. Lin, D. Sun, W. Huo, and C. Zhang, "A large-scale empirical study on vulnerability distribution within projects and the lessons learned," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 1547–1559.

[33] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, and F. Massacci, "Vuln4real: A methodology for counting actually vulnerable dependencies," *IEEE Transactions on Software Engineering*, vol. 48, no. 5, pp. 1592–1609, 2020.

[34] C. Xu, B. Chen, C. Lu, K. Huang, X. Peng, and Y. Liu, "Tracking patches for open source software vulnerabilities," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 860–871.

[35] Y. Zhou, J. K. Siow, C. Wang, S. Liu, and Y. Liu, "Spi: Automated identification of security patches via commits," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–27, 2021.

[36] X. Tan, Y. Zhang, C. Mi, J. Cao, K. Sun, Y. Lin, and M. Yang, "Locating the security patches for disclosed oss vulnerabilities with vulnerability-commit correlation ranking," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 3282–3299.

[37] S. Wang, Y. Zhang, L. Bao, X. Xia, and M. Wu, "Vcmatch: a ranking-based approach for automatic security patches localization for oss vulnerabilities," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2022, pp. 589–600.

[38] T. Dunlap, E. Lin, W. Enck, and B. Reaves, "Vfcfinder: Pairing security advisories and patches," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1128–1142.

[39] J. Yu, Y. Chen, D. Tang, X. Liu, X. Wang, C. Wu, and H. Tang, "Llm-enhanced software patch localization," *arXiv preprint arXiv:2409.06816*, 2024.

[40] FiRST, "CVSS," https://www.first.org/cvss/, 2025.

[41] Z. Han, X. Li, Z. Xing, H. Liu, and Z. Feng, "Learning to predict severity of software vulnerability using only vulnerability description," in *2017 IEEE International conference on software maintenance and evolution (ICSME)*. IEEE, 2017, pp. 125–136.

[42] C. Elbaz, L. Rilling, and C. Morin, "Fighting n-day vulnerabilities with automated cvss vector prediction at disclosure," in *Proceedings of the 15th International Conference on Availability, Reliability and Security*, 2020, pp. 1–10.

[43] T. H. M. Le, D. Hin, R. Croft, and M. A. Babar, "Deepcva: Automated commit-level vulnerability assessment with deep multi-task learning," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 717–729.

[44] T. H. M. Le and M. A. Babar, "On the use of fine-grained vulnerable code statements for software vulnerability assessment models," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 621–633.

[45] E. Aghaei, E. Al-Shaer, W. Shadid, and X. Niu, "Automated cve analysis for threat prioritization and impact prediction," *arXiv preprint arXiv:2309.03040*, 2023.

[46] S. Pan, L. Bao, J. Zhou, X. Hu, X. Xia, and S. Li, "Towards more practical automation of vulnerability assessment," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[47] S. Pan, L. Bao, X. Xia, D. Lo, and S. Li, "Fine-grained commit-level vulnerability type prediction by cwe tree structure," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 957–969.

[48] M. Fu, V. Nguyen, C. K. Tantithamthavorn, T. Le, and D. Phung, "Vulexplainer: A transformer-based hierarchical distillation for explaining vulnerability types," *IEEE Transactions on Software Engineering*, vol. 49, no. 10, pp. 4550–4565, 2023.

[49] X.-C. Wen, C. Gao, F. Luo, H. Wang, G. Li, and Q. Liao, "Livable: exploring long-tailed classification of software vulnerability types," *IEEE Transactions on Software Engineering*, 2024.

[50] Y. Luo, W. Xu, and D. Xu, "Predicting code vulnerability types via heterogeneous gnn learning," in *European Symposium on Research in Computer Security*. Springer, 2024, pp. 67–87.

[51] C. Ji, S. Yang, H. Sun, and Y. Zhang, "Applying contrastive learning to code vulnerability type classification," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 11 942–11 952.

[52] M. Bozorgi, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond heuristics: learning to classify vulnerabilities and predict exploits," in *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2010, pp. 105–114.

[53] C. Sabottke, O. Suciu, and T. Dumitraș, "Vulnerability disclosure in the age of social media: Exploiting twitter for predicting {Real-World} exploits," in *24th USENIX security symposium (USENIX security 15)*, 2015, pp. 1041–1056.

[54] N. Tavabi, P. Goyal, M. Almukaynizi, P. Shakarian, and K. Lerman, "Darkembed: Exploit prediction with neural language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[55] H. Chen, R. Liu, N. Park, and V. Subrahmanian, "Using twitter to predict when vulnerabilities will be exploited," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data Mining*, 2019, pp. 3143–3152.

[56] J. Jacobs, S. Romanosky, I. Adjerid, and W. Baker, "Improving vulnerability remediation through better exploit prediction," *Journal of Cybersecurity*, vol. 6, no. 1, p. tyaa015, 2020.

[57] J. Jacobs, S. Romanosky, B. Edwards, I. Adjerid, and M. Roytman, "Exploit prediction scoring system (epss)," *Digital Threats: Research and Practice*, vol. 2, no. 3, pp. 1–17, 2021.

[58] O. Suciu, C. Nelson, Z. Lyu, T. Bao, and T. Dumitraș, "Expected exploitability: Predicting the development of functional vulnerability exploits," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 377–394.

[59] J. Jacobs, S. Romanosky, O. Suciu, B. Edwards, and A. Sarabi, "Enhancing vulnerability prioritization: Data-driven exploit predictions with community-driven insights," in *2023 IEEE European symposium on security and privacy workshops (euroS&pW)*. IEEE, 2023, pp. 194–206.

[60] MVN, "MCR," https://mvnrepository.com/repos/central, 2025.

[61] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu, Z. Wu, T. Liu *et al.*, "A survey on in-context learning," *arXiv preprint arXiv:2301.00234*, 2022.

[62] R. C. Martin, "Acyclic dependencies principle," https://en.wikipedia.org/wiki/Acyclic_dependencies_principle, 2015.

[63] Maven, "Maven Central: dom4j:dom4j:1.5.2," https://central.sonatype.com/artifact/dom4j/dom4j/1.5.2, 2005.

[64] ——, "Maven Central: jaxen:jaxen:1.1-beta-4," https://central.sonatype.com/artifact/jaxen/jaxen/1.1-beta-4, 2005.

[65] NVD, "NVD - CVE-2016-5393 — nvd.nist.gov," https://nvd.nist.gov/vuln/detail/CVE-2016-5393, 2021.

[66] R. Vallée-Rai, P. Co, E. Gagnon, L. Hendren, P. Lam, and V. Sundaresan, "Soot: A java bytecode optimization framework," in *CASCON First Decade High Impact Papers*, 2010, pp. 214–224.

[67] NIST, "NVD - Home," https://nvd.nist.gov/, 2024.

[68] T. Lee, "Pull request," https://github.com/line/armeria/commit/e2697a575e9df6692b423e02d731f293c1313284, 2021.

[69] gtully, "Pull request," https://github.com/apache/activemq-artemis/commit/7dd50bd58ed2b46aa7ecdba7cffe979b672d58fb, 2020.

[70] NVD, "NVD - CVE-2021-43795," https://nvd.nist.gov/vuln/detail/CVE-2021-43795, 2021.

[71] ——, "NVD - CVE-2021-26118," https://nvd.nist.gov/vuln/detail/CVE-2021-26118, 2021.

[72] B. Ruan, J. Liu, C. Zhang, and Z. Liang, "Kernjc: Automated vulnerable environment generation for linux kernel vulnerabilities," *arXiv preprint arXiv:2404.11107*, 2024.

[73] L. Bao, X. Xia, A. E. Hassan, and X. Yang, "V-szz: automatic identification of version ranges affected by cve vulnerabilities," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 2352–2364.

[74] S. Wu, R. Wang, K. Huang, Y. Cao, W. Song, Z. Zhou, Y. Huang, B. Chen, and X. Peng, "Vision: Identifying affected library versions for open source software vulnerabilities," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1447–1459.

[75] Y. Cheng, L. K. Shar, T. Zhang, S. Yang, C. Dong, D. Lo, S. Lv, Z. Shi, and L. Sun, "Llm-enhanced static analysis for precise identification of vulnerable oss versions," *arXiv preprint arXiv:2408.07321*, 2024.

[76] S. Sun, Y. Xing, X. Wang, S. Wang, Q. Li, and K. Sun, "Dispatch: unraveling security patches from entangled code changes," in *Proceedings of the 34th USENIX Conference on Security Symposium*, 2025.

[77] Y. Li, T. Tan, and J. Xue, "Understanding and analyzing java reflection," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 28, no. 2, pp. 1–50, 2019.

[78] X. Song, Y. Wang, X. Cheng, G. Liang, Q. Wang, and Z. Zhu, "Efficiently trimming the fat: Streamlining software dependencies with java reflection and dependency analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.

[79] thebjorn, "thebjorn/pydeps," https://github.com/thebjorn/pydeps, 2025.

[80] V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos, "Pycg: Practical call graph generation in python. in 2021 ieee/acm 43rd international conference on software engineering (icse)," *IEEE, IEEE, Madrid, Spain*, pp. 1646–1657, 2021.

[81] E. Iannone, D. Di Nucci, A. Sabetta, and A. De Lucia, "Toward automated exploit generation for known vulnerabilities in open-source libraries," in *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. IEEE, 2021, pp. 396–400.

[82] H. J. Kang, T. G. Nguyen, B. Le, C. S. Păsăreanu, and D. Lo, "Test mimicry to assess the exploitability of library vulnerabilities," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 276–288.

[83] Z. Zhou, Y. Yang, S. Wu, Y. Huang, B. Chen, and X. Peng, "Magneto: A step-wise approach to exploit vulnerabilities in dependent libraries via llm-empowered directed fuzzing," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1633–1644.

[84] Z. Chen, X. Hu, X. Xia, Y. Gao, T. Xu, D. Lo, and X. Yang, "Exploiting library vulnerability via migration based automating test generation," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.

[85] B. Ruan, J. Liu, W. Zhao, and Z. Liang, "Vulzoo: A comprehensive vulnerability intelligence dataset," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 2334–2337.